Estudo Orientado
Modelo Lógico de Banco de Dados

fantini@coltec.ufmg

19 de outubro de 2026

Sumário

1 Modelo Lógico - Introdução **3**

1.1 Introdução - Modelagem de Banco de Dados 3

1.2 Introdução ao Modelo Lógico 3

1.2.1 Diferentes Modelos Lógicos 4

1.2.2 Mapeamento Modelo Conceitual - Lógico 4

2 Abordagem ER e Relacional **5**

3 Modelo Lógico - Conceitos **5**

3.1 Revendo os Conceitos do Banco Relacional 6

3.1.1 Conceito de Chave 7

3.2 Chave Primária e Chave Estrangeira 7

3.2.1 Chave Estrangeira 8

3.3 Chave Alternativa 10

4 Valor NULL - Domínios e Valores Vazios **11**

4.1 Do Modelo Conceitual para o Lógico 11

4.1.1 Evitar junções 12

4.1.2 Diminuir o número de chaves primárias 12

4.1.3 Evitar campos opcionais 12

4.2 Etapas na Modelagem Conceitual → Lógico 13
4.2.1 Etapa 1 - Mapeamento das Entidades 14
4.2.2 Dicas para Nome das Tabelas e Campos 15
4.3 Etapa 2 - Entidade Fraca 15
4.3.1 Etapa 3 - Atributos Compostos 16
4.3.2 Etapa 4 - Atributo multivalorado 17
4.4 Etapa 5- Mapeando Relacionamentos 18

5 Exemplos de Mapeamento de Relacionamentos 19
5.1 Mapeamento de Relacionamento 1:1 19
5.2 Mapeamento de Relacionamento 1:N 22
5.3 Mapeamento de relacionamento N:N 25

6 Exercícios 28

7 Referências 29

1 Modelo Lógico - Introdução

1.1 Introdução - Modelagem de Banco de Dados

Como explicado na nossa introdução ao projeto de banco de dados, a modelagem do banco de dados é crucial para obtermos um banco de dados com uma estrutura adequada.

Algumas características e conceitos da modelagem devem ser bem compreendidos:

- Modelagem de dados é o processo de planejamento de uma estrutura para representar como as informações e os relacionamentos entre as informações serão armazenados em um sistema de gerenciamento de dados.
- A modelagem de dados é um processo que ajuda a compreender as estruturas, a forma e os relacionamentos das informações. Tal como qualquer modelo, é uma versão simplificada e imperfeita da realidade.
- A modelagem é um processo que faz uso de três tipos progressivos de modelos de dados: modelagem de dados conceituais, modelagem de dados lógicos e modelagem de dados físicos.
- Cada etapa da modelagem de um banco de dados trata um aspecto importante para a construção final de um modelo de dados útil para o problema proposto.
- Um modelo de dados conceitual é o nível mais alto e, portanto, o menos detalhado.
- Um modelo lógico de dados envolve um pensamento mais detalhado sobre a implementação sem realmente implementar nada.
- Finalmente, o modelo de dados físicos baseia-se nos requisitos do modelo de dados lógicos para criar um banco de dados real.

1.2 Introdução ao Modelo Lógico

Um modelo lógico de dados é a segunda etapa no processo de modelagm de banci de dados.

Esta etapa envolve o preenchimento dos detalhes do modelo conceitual. Ainda é muito cedo para escolher um SGBD específico, mas esta etapa pode ajudá-lo a decidir qual classe de banco de dados usar (relacional, documento, etc).

Por exemplo, se você decidir relacional, então é hora de decidir quais tabelas criar. Se você decidir o documento, então é hora de definir as coleções.

Decida também os detalhes de cada campo/coluna individual e relacionamento. Isso inclui tipos de dados, tamanhos, comprimentos, matrizes, objetos aninhados, etc.

O modelo lógico normalmente é criado por arquitetos e analistas de negócios.

Nesse estudos consideramos que vamos usar um SGBD relacional, ou seja, nosso modelo lógico já tratará os aspectos de um SGBD relacional, que lida com tabelas correlacionadas.

Vamos estudar o **mapeamento do modelo ER para o modelo lógico (que no nosso caso será o modelo relacional).**

O seja, para essa etapa temos que ter concluido o modelo conceitual.

1.2.1 Diferentes Modelos Lógicos

O modelo lógico relacional é apenas um tipo, dentre vários, como mostrado na figura 1.

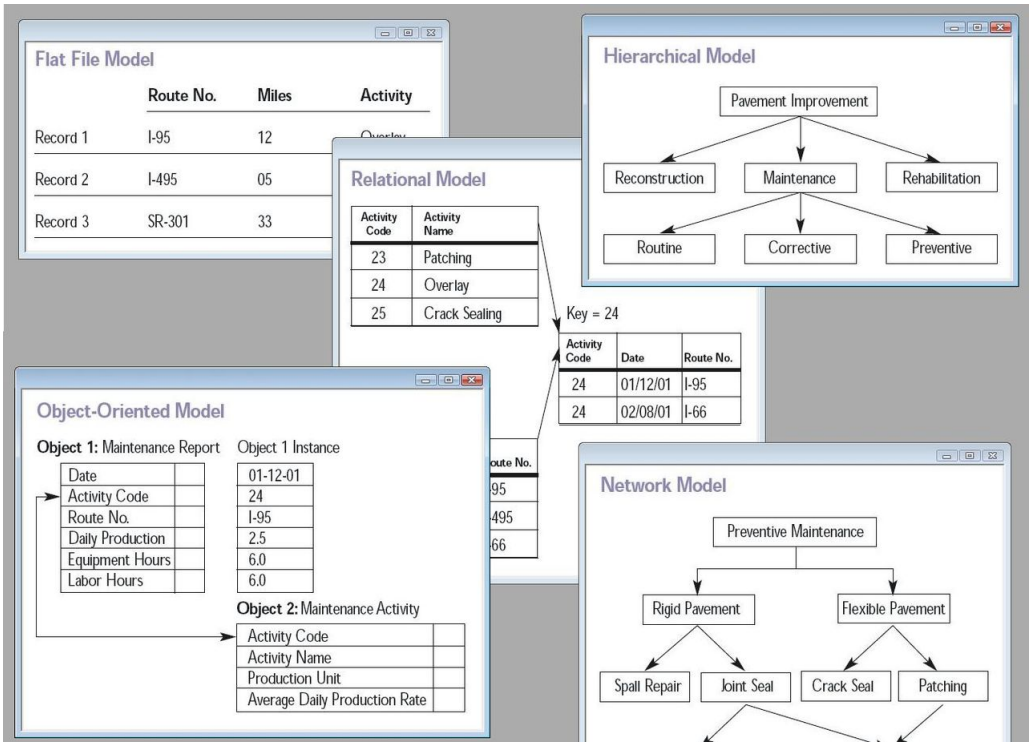


Figura 1: Diferentes Modelos Lógicos

1.2.2 Mapeamento Modelo Conceitual - Lógico

Na figura 2 apresentamos um diagrama exemplificando o fluxo do projeto, inicinando com o modelo conceitual, passando pelo relacional (no caso, já definido que o banco de dados será relacional).

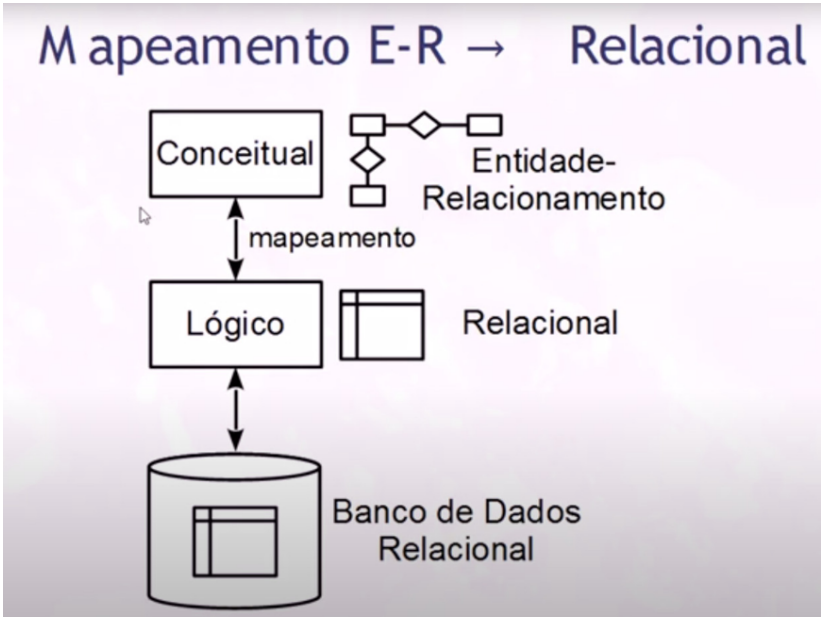


Figura 2: Mapeamentodo modelo conceitual na abordagem ER para modelo lógico Relacional

2 Abordagem ER e Relacional

A abordagem ER vista no estudo anterior, é voltada à modelagem de dados de forma independente do SGBD considerado e é adequada para construção do modelo conceitual.

Já a **abordagem relacional** modela os dados a nível de **SGBD relacional**.

Um modelo neste nível de abstração é chamado de **modelo lógico**.

O modelo lógico consiste na especificação lógica (logicamente...) da estrutura do banco de dados. **Ele já aproxima o modelo conceitual da realidade da implementação no SCBD**.

Por exemplo, na fase de projeto lógico os tipos de dados devem ser detalhados e as relações devem ser explicitadas. Ou seja, nesse nível da modelagem, já fazemos a modelagem pensando no SGBD que será usado.

Lembre-se que o banco de dados relacional, usando tabelas, **é uma opção**, dentre várias, para a escolha do tipo de banco de dados.

Podemos ter outros tipos de modelos lógicos, associados a outros tipos de banco de dados, como por exemplo o modelo lógico hierárquico (que usa estrutura de árvores e faz uso por exemplo das linguagens JSON/XML) ou o modelo de grafos.

(A figura 1 acima apresenta alguns modelos lógicos de banco de dados).

3 Modelo Lógico - Conceitos

Nesse roteiro de estudos vamos estudar o **projeto lógico de Banco de Dados relacional**.

Como dito, abordaremos o **mapeamento do modelo conceitual construído na abordagem ER para o modelo lógico relacional**, que implementa, a nível de SGBD relacional, os dados representados abstratamente no modelo ER.

O termo "implementação" significa que ocorre uma transformação de um modelo mais abstrato para um modelo que contém mais detalhes de implementação.

Um determinado modelo ER pode ser implementado através de diversos modelos relacionais, que contém as informações especificadas pelo diagrama ER. Todos podem ser considerados uma implementação correta do modelo ER considerado.

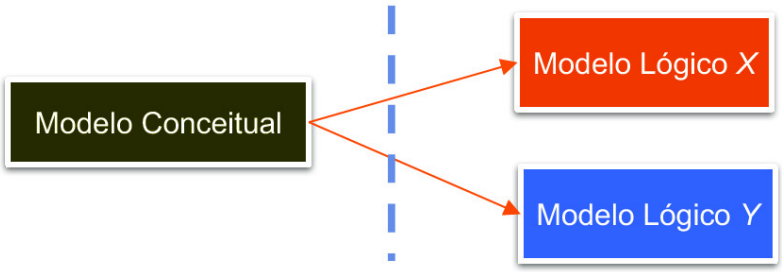


Figura 3: Um modelo conceitual pode derivar diferentes modelos lógicos

Entretanto, estes diferentes modelos relacionais podem resultar em diferentes performances do sistema construído sobre o banco de dados e podem implicar maior facilidade, ou dificuldade de desenvolvimento e manutenção do sistema construído sobre o banco de dados. Vide figura 3 na qual mostramos um diagrama representando um modelo conceitual que pode ser convertido em dois modelos lógicos diferentes.

Geralmente fazemos um modelo lógico inicial e caso esse modelo não atenda aos requisitos de performance da BD projetada, há um processo de refinamento e melhoria do modelo, até ser atingido o modelo relacional satisfatório.

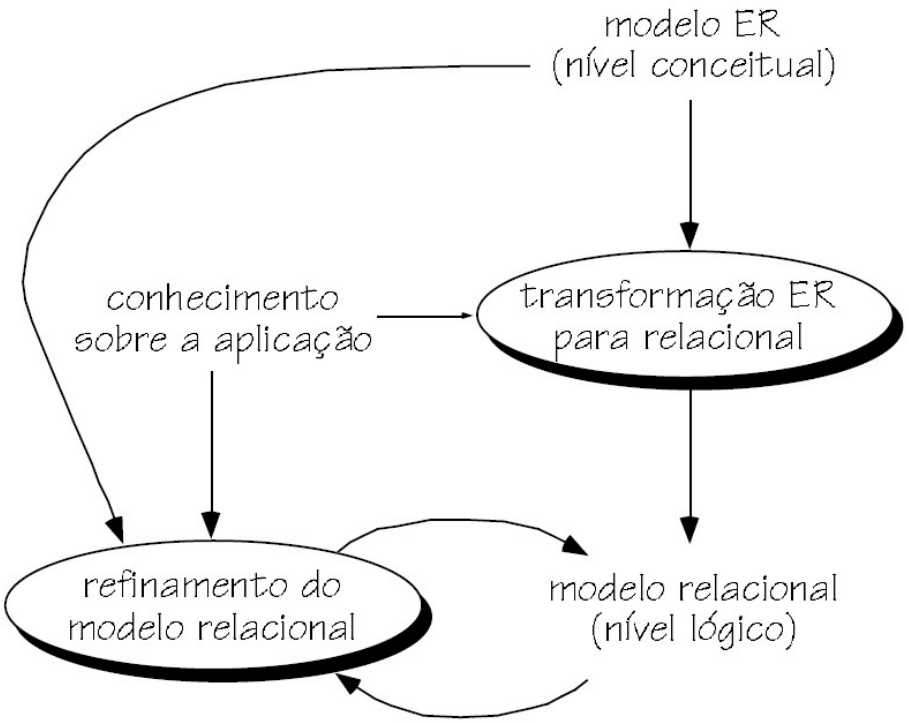


Figura 4: Diagrama mostrando o processo de criação do modelo Lógico

3.1 Revendo os Conceitos do Banco Relacional

Principais Conceitos

- Um banco de dados relacional é composto de tabelas ou relações.
- A terminologia tabela é mais comum nos produtos comerciais e na prática.
- Já a terminologia relação foi utilizada na literatura original sobre a abordagem relacional (daí a denominação "relacional") e é mais comum na área acadêmica e nos livros texto.

Tabela (vide figura 5)

- Uma tabela é um conjunto não ordenado de linhas (tuplas, na terminologia acadêmica).
- Cada linha é composta por uma série de campos (valor de atributo, na terminologia acadêmica).
- Cada campo é identificado por nome de campo (nome de atributo, na terminologia acadêmica).
- O conjunto de campos das linhas de uma tabela que possuem o mesmo nome formam uma coluna.

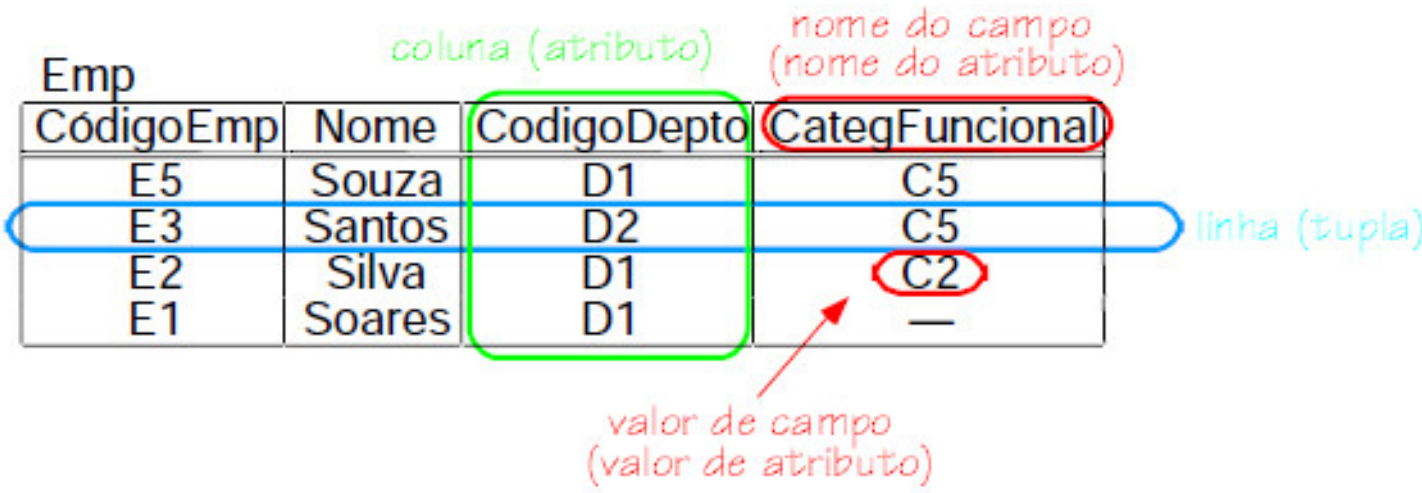


Figura 5: Conceituação (formal) de uma tabela

3.1.1 Conceito de Chave

O conceito básico para estabelecer relações entre linhas de tabelas de um banco de dados relacional é o da **chave**.

Em um banco de dados relacional, há ao menos três tipos de chaves a considerar:

- chave primária
- chave estrangeira,
- chave alternativa\.

3.2 Chave Primária e Chave Estrangeira

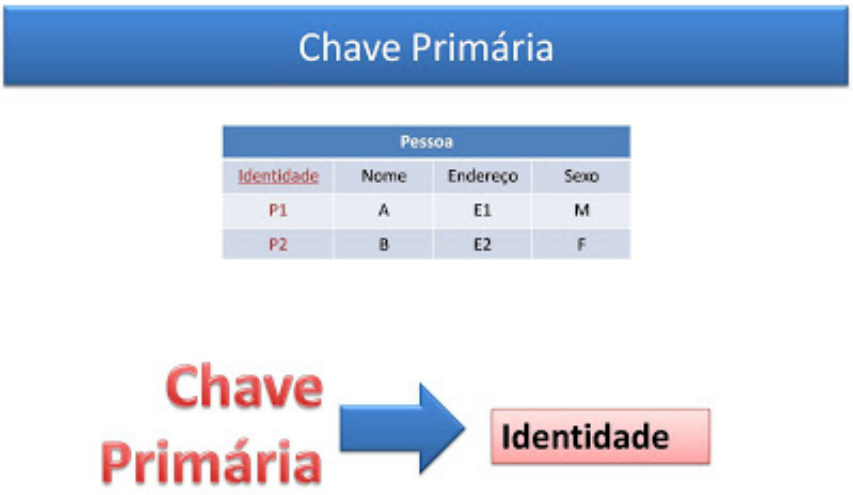


Figura 6: Exemplo de chave primária

Uma chave primária é uma coluna ou uma combinação de colunas cujos valores distinguem uma linha das demais dentro de uma tabela.

Na tabela **Emp** da Figura 5 a chave primária é a coluna CódigoEmp.

Na figura 6 a chave primária é a coluna ”identidade”.

Na Figura 7 apresenta-se um exemplo de uma tabela (Dependente) que possui uma chave primária composta (colunas CódigoEmp e NoDepen).

Neste caso, apenas um dos valores dos campos que compõem a chave não é suficiente para distinguir uma linha das demais, já que tanto um código de empregado (CódigoEmp) pode aparecer em diferentes linhas, quanto um número de dependente (NoDepen) pode aparecer em diferentes linhas.

É necessário considerar ambos valores (CódigoEmp e NoDepen) para identificar uma linha na tabela, ou seja para identificar um dependente

No exemplo da tabela da Figura 7, qualquer combinação de colunas que contenha as colunas CódigoEmp e NoDepen é uma chave primária.

Dependente

CódigoEmp	NoDepen	Nome	Tipo	DataNasc
E1	01	João	Filho	12/12/91
E1	02	Maria	Esposa	01/01/50
E2	01	Ana	Esposa	05/11/55
E5	01	Paula	Esposa	04/07/60
E5	02	José	Filho	03/02/85

Figura 7: Exemplo de chave primária composta

Por isso, nas definições formais de chave primária, exige-se que essa seja **mínima**. Uma chave é mínima quando todas suas colunas forem efetivamente necessárias para garantir o requisito de unicidade de valores da chave.

Exemplificando, alguém poderia considerar a combinação de colunas CódigoEmp, NoDepen e Tipo como sendo uma chave primária. Entretanto, se eliminarmos, desta combinação a coluna Tipo continuamos frente a uma chave primária. Portanto, a combinação de colunas CódigoEmp, NoDepen e Tipo não obedece o princípio da minimalidade e não deve ser considerada uma chave.

3.2.1 Chave Estrangeira

Uma **chave estrangeira** é uma coluna ou uma combinação de colunas, cujos valores aparecem necessariamente na chave primária de uma tabela.

A chave estrangeira é o mecanismo que permite a implementação de relacionamentos em um banco de dados relacional.

Nas colunas da Figura 8 a coluna **CodigoDepto** da tabela Emp é uma chave estrangeira em relação a chave primária da tabela Dept.

Isso significa que, na tabela Emp, não podem aparecer linhas que contenham um valor do campo CodigoDepto que não exista na coluna de mesmo nome da tabela Emp. A interpretação desta restrição é **que todo empregado deve estar associado a um departamento**.

Dept

CodigoDepto	NomeDepto
D1	Compras
D2	Engenharia
D3	Vendas

Emp

CodigoEmp	Nome	CodigoDepto	CategFuncional	CIC
E1	Souza	D1	-	132.121.331-20
E2	Santos	D2	C5	891.221.111-11
E3	Silva	D2	C5	341.511.775-45
E5	Soares	D1	C2	631.692.754-88

Figura 8: Exemplo de Chave Estrangeira: a coluna CodigoDepto da tabela Dept é a chave primária. A coluna CodigoDepto da tabela Emp é uma chave estrangeira em relação a chave primária da tabela Dept.

Importante A palavra "estrangeira" usada para denominar este tipo de chave pode ser enganosa.

Ela pode levar a crer que a chave estrangeira sempre referencia uma chave primária de outra tabela.

Entretanto, esta restrição não existe. Uma chave primária pode referenciar a chave primária da própria tabela, como mostra a Figura 9.

Emp

CódigoEmp	Nome	CodigoDepto	CodigoEmpGerente
E5	Souza	D1	—
E3	Santos	D2	E5
E2	Silva	D1	E5
E1	Soares	D1	E1

chave estrangeira:
referencia a chave primária
da própria tabela

Figura 9: Exemplo de Chave Estrangeira dentro da própria tabela.

Na tabela da figura 9 a coluna **CodigoEmpGerente** é o código de um outro empregado, o gerente do empregado correspondente a linha em questão.

Como todo gerente é ele mesmo também um empregado, existe a restrição de que todo valor da coluna **CodigoEmpGerente** deve aparecer na coluna **CodigoEmp**.

Assim, a coluna **CodigoEmpGerente** é chave estrangeira em relação a chave primária da própria tabela Emp.

3.3 Chave Alternativa

Em alguns casos, mais de uma coluna ou combinações de colunas podem servir para distinguir uma linha das demais.

Uma das colunas (ou combinação de colunas) é escolhida como chave primária. As demais colunas ou combinações são denominadas chaves alternativas.

Emp

CódigoEmp	Nome	CodigoDepto	CategFuncional	CIC
E5	Souza	D1	C5	132.121.331-20
E3	Santos	D2	C5	891.221.111-11
E2	Silva	D1	C2	341.511.773-45
E1	Soares	D1	—	631.692.754-88

Figura 10: Chave alternativa: A coluna CIC é uma chave alternativa.

A Figura 10 mostra um exemplo de uma tabela com dados de empregados (Emp) na qual tanto a coluna CódigoEmp quanto a coluna CIC podem ser usadas para distinguir uma linha das demais. Nesta tabela, como a coluna CódigoEmp foi escolhida como chave primária, diz-se que a coluna CIC é uma chave alternativa.

Importante

Quando, em uma tabela, mais de uma coluna ou combinações de colunas podem servir para distinguir uma linha das demais, surge a questão de que critério deve-se usar para determinar qual das possíveis colunas (ou combinação de colunas) será usada como chave primária.

No exemplo da figura 10 a questão é que critério foi usado para preferir a coluna **CódigoEmp** como chave primária e considerar a coluna **CIC** como chave alternativa?

Porque CIC não foi usado como chave primária e CódigoEmp como chave alternativa?

Geralmente, se considerarmos apenas a tabela em que a coluna aparece, não há diferença entre uma coluna ser chave primária ou alternativa.

Em ambos casos, apenas está sendo especificada a unicidade de valores de chave.

Entretanto, ao considerarmos chaves estrangeiras, a diferenciação entre chave primária e chave alternativa passa a ser relevante.

Quando especificamos que uma chave é primária, estamos especificando, além da unicidade de valores, também o fato de esta coluna ser usada nas chaves estrangeiras que referenciam a tabela em questão.

Assim, no caso da tabela da Figura 10, estamos especificando que tanto os valores de **Códi goEmp** quanto os valores de **>CIC** são únicos e adicionalmente que a coluna CódigoEmp será usada nas chaves estrangeiras que referenciam a tabela Emp.

4 Valor NULL - Domínios e Valores Vazios

Quando uma tabela do banco de dados é definida, para cada coluna da tabela, deve ser especificado um conjunto de valores (alfanumérico, numérico,...) que os campos da respectiva coluna podem assumir.

Este conjunto de valores é chamado de **domínio da coluna** ou **domínio do campo**.

Além disso, deve ser especificado se os campos da coluna podem estar vazios ("null" em inglês) ou não.

Estar vazio indica que o campo não recebeu nenhum valor de seu domínio.

Na Figura 10, o campo **CategFuncional** da linha correspondente ao empregado de código E1 está vazio.

Isso indica que o empregado E1 não possui categoria funcional ou que esta ainda não foi informada.

As colunas nas quais não são admitidos valores vazios são chamadas de colunas obrigatórias.

As colunas nas quais podem aparecer campos vazios são chamadas de colunas opcionais.

Normalmente, os SGBD relacional exigem que todas colunas que compõem a chave primária sejam obrigatórias.

A mesma exigência não é feita para as demais chaves.

4.1 Do Modelo Conceitual para o Lógico

Vamos apresentar as regras para transformação de um modelo ER em um modelo relacional.

As regras foram definidas tendo em vista dois objetivos básicos:

- Obter um banco de dados que permita boa performance de instruções de consulta e alteração do banco de dados. Obter boa performance significa basicamente diminuir o número de acessos a disco, já que estes consomem o maior tempo na execução de uma instrução de banco de dados.
- Obter um banco de dados que simplifique o desenvolvimento e a manutenção de aplicações.

Estes dois objetivos são os centrais. Além destes, as regras de transformação procuram obter um banco de dados que ocupe pouco espaço em disco.

O objetivo de reduzir espaço de armazenamento era até algum tempo atrás tão importante quanto o objetivo de melhorar performance e simplificar o desenvolvimento.

Entretanto, nos últimos anos, o preço dos meios de armazenamento vem diminuindo constantemente, o que fez com que a redução de espaço ocupado, principalmente se em detrimento dos demais objetivos, diminuísse de importância.

Afim de alcançar estes objetivos, as regras de tradução foram definidas tendo por base, entre outros, os seguintes princípios:

- Evitar junções
- Diminuir o número de chaves primárias
- Evitar campos opcionais

4.1.1 Evitar junções

Ter os dados necessários a uma consulta em uma única linha Um SGBD relacional normalmente armazena os dados de uma linha de uma tabela contiguamente em disco.

Com isso, todos dados de uma linha são trazidos para a memória em uma operação de acesso a disco. Isso significa que, uma vez encontrada uma linha de uma tabela, seus campos estão todos disponíveis sem necessidade de acesso adicionais a disco.

Quando for necessário buscar em disco dados de diversas linhas associadas pela igualdade de campos (por exemplo, buscar os dados de um empregado e os dados de seu departamento) é necessário usar a operação de junção. Os SGBD procuram implementar a junção de forma eficiente, já que ela é uma operação executada muito freqüentemente. Mesmo assim, a junção envolve diversos acessos a disco. Assim, quando for possível, é preferível ter os dados necessários a uma consulta em uma única linha somente, ao invés de tê-los distribuídos em diversas linhas, exigindo a sua junção.

4.1.2 Diminuir o número de chaves primárias

Para a implementação eficiente do controle da unicidade da chave primária, o SGBD usa normalmente uma estrutura de acesso auxiliar, um índice, para cada chave primária. Índices, pela forma que são implementados, tendem a ocupar espaço considerável em disco.

Além disso, a inserção ou remoção de entradas em um índice podem exigir diversos acesso a disco. Assim sendo, quando for necessário escolher entre duas alternativas de implementação, uma na qual dados aparecem em uma única tabela e outra na qual os mesmos dados aparecem em duas ou mais tabelas com a mesma chave primária e mesmo número de linhas, a implementação por uma única tabela deve ser preferida.

Para exemplificar vamos considerar que se deseja armazenar dados sobre clientes em um banco de dados relacional. Deseja-se armazenar, para cada cliente, seu código, seu nome, o nome da pessoa de contato, o endereço e o telefone. Este dados poderiam ser implementados através de uma das seguintes alternativas:

4.1.3 Evitar campos opcionais

Campos opcionais são campos que podem assumir o valor VAZIO (NULL em SQL).

Os SGBD relacionais usualmente não desperdiçam espaço pelo fato de campos de uma linha estarem vazios, pois usam técnicas de compressão de dados e registros de tamanho variável no armazenamento interno de linhas.

Além disso, há uma cláusula de SQL que especifica ao SGBD se o campo deve estar preenchido ou pode estar vazio.

Assim, em princípio, não há problemas em usar este tipo de campos. Uma situação que pode gerar problemas é aquela na qual a obrigatoriedade ou não do preenchimento de um campo depende do valor de outros campos.

Neste caso, em alguns SGBD, o controle da obrigatoriedade deve ser feito pelos programas que acessam o banco de dados, o que deve ser evitado.

4.2 Etapas na Modelagem Conceitual → Lógico

Nessa seção descrevemos as ações mais comuns que temos que fazer para passar o modelo conceitual (diagrama DER) para o modelo lógico (no caso do modelo RELACIONAL, ele será construido em tabelas).

A transformação de um modelo ER em um modelo relacional dá-se nos seguintes passos:

1. Tradução inicial de entidades e respectivos atributos
2. Tradução de relacionamentos e respectivos atributos
3. Tradução de generalizações/especializações

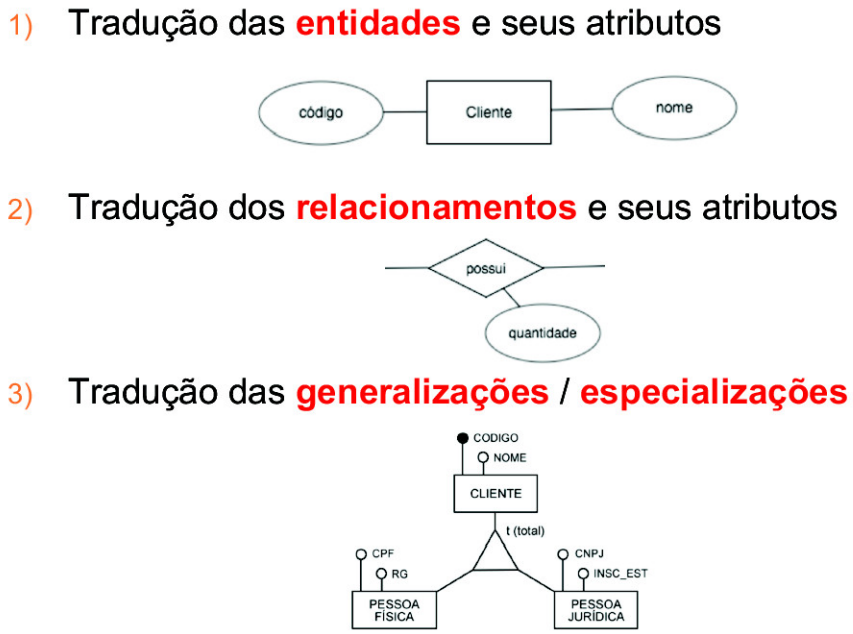


Figura 11: Resumo das propriedades de conversão do modelo conceitual no lógico.

Comentário Trata-se aqui de uma tradução inicial. Pelas regras que seguem nas próximas seções, as tabelas definidas nesta etapa ainda poderão ser fundidas, no caso de algumas alternativas de implementação de relacionamentos 1:1 e hierarquias de generalização/especialização.

4.2.1 Etapa 1 - Mapeamento das Entidades

Esse passo é razoavelmente óbvio:

- cada entidade é traduzida para uma tabela.
- cada atributo da entidade define uma coluna da tabela.
- Os atributos identificadores da entidade correspondem às colunas que compõem a chave primária da tabela.

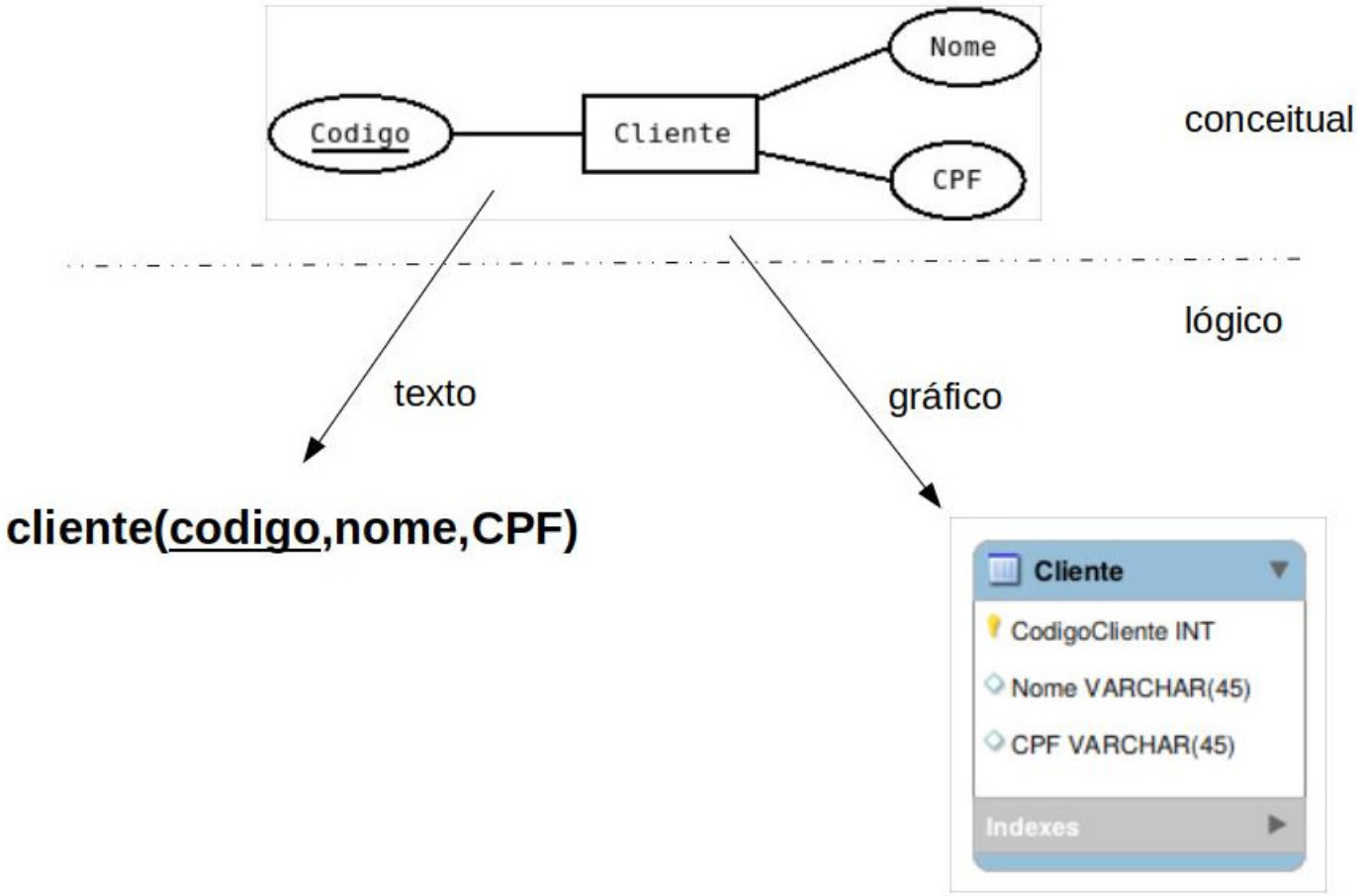


Figura 12: Exemplo de entidade e seus atributos formando uma tabela

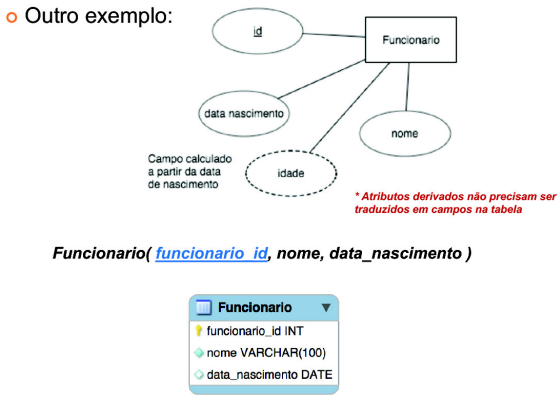


Figura 13: Exemplo de parte de um DER com entidade possuindo atributos derivados

4.2.2 Dicas para Nome das Tabelas e Campos

- Não usar espaços em branco
- Não usar caracteres especiais ou acentuados
- Abreviar quando possível

Entidade ou Atributo	Tabela ou Campo
Itens de Venda	*Itens_venda*
Data de Nascimento	*data_nascimento* ou *dt_nascimento*
Endereço	*endereco*
% de desconto	*porcentagem_desconto* ou *porc_desc*
Seção	*Secao*

Figura 14: Exemplo de nomes para as colunas das tabelas

4.3 Etapa 2 - Entidade Fraca

Cada entidade fraca deve ser transformada em uma tabela

Incluir os atributos da chave primária da tabela dominante como chave estrangeira.

A chave primária da relação deve ser a combinação dos atributos da chave primária da relação dominante e da chave da entidade fraca.

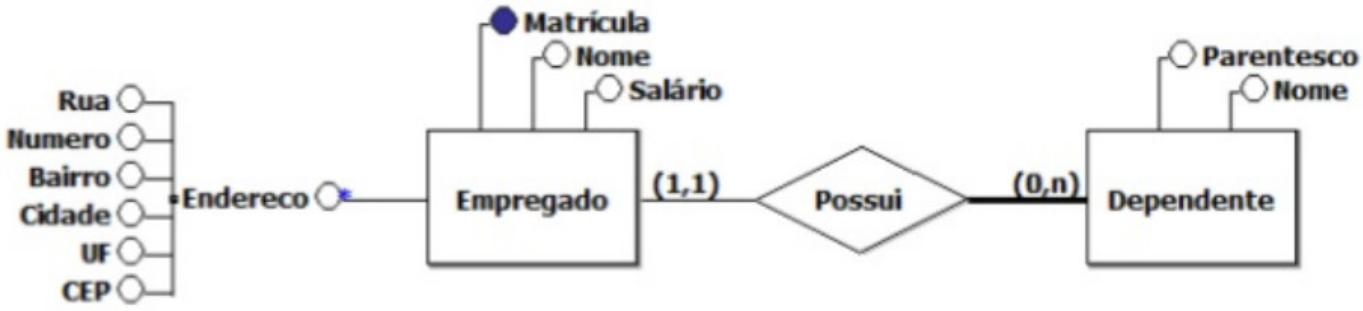


Figura 15: Relacionamento identificador (entidade DEPENDENTE é uma "entidade fraca"). A entidade DEPEDENTE será uma tabela no modelo lógico, com relação de chaves (chave estrangeira-chave primária) com a entidade (tabela) EMPREGADO.

No exemplo da figura 15 a entidade fraca dependente passa a ser uma tabela, com a chave estrangeria sendo composta pela chave do estrangeria da entidade empregado mais a chave primária da tabela dependente, como mostrado na Figura 16 (no formato texto).

- Empregado (Matrícula, Nome, Salário, Rua, Número, Bairro, Cidade, UF, CEP);
- Dependente (MatriculaEmpregado, Nome, Parentesco)

Figura 16: Descrição das tabelas derivadas a partir do DER da Figura **??**

Regra para Mapear Entidade Fraca

- Para cada tipo de entidade fraca **F** no modelo conceitual: com tipo de entidade proprietária forte **F**, crie uma tabela **T** e inclua todos os atributos simples (ou componentes simples dos atributos compostos) de **F** como atributos de **T**.
- Além disso, inclua como atributos chave estrangeira de **T** os atributos de chave primária da tabela que correspondem aos de entidade proprietária **E**.
- Assim mapeamos a identificação da entidade fraca **F**.
- A chave primária de **T** é a combinação das chaves primárias da entidade proprietária **E** e a chave parcial do tipo de entidade fraca **F**, se houver

4.3.1 Etapa 3 - Atributos Compostos

Como dito acima, a transformação mais natural é um entidade virar tabela e seus atributos virarem colunas dessas tabelas.

Quando temos atributos compostos, podemos criar uma nova tabela e associar os dados dela a partir de chave primária-estrangeira (mais usado quando o atributo é multivalorado, ou seja, podem ter mais de um valor) ou criar os campos na mesma tabela (mais recomendado).

Nesse segundo caso, podemos, para facilitar a identificação, colocar como label do campo o nome do atrituto composto, colocando em seguida os atributos que o definem. Veja a figura 17 abaixo:

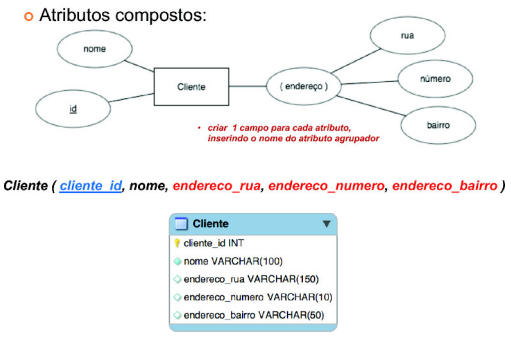


Figura 17: Exemplo de atributo composto. Pode-se criar campos na tabela fazendo referência ao atributo (e suas especificidades) ou pode-se criar uma tabela específica (menos comum) para os atributos compostos

4.3.2 Etapa 4 - Atributo multivalorado

Nesse caso a recomendação é que seja criada uma tabela específica para o atributo (que poderá ter mais de um valor para cada instância da entidade). Veja a figura 18 a seguir.

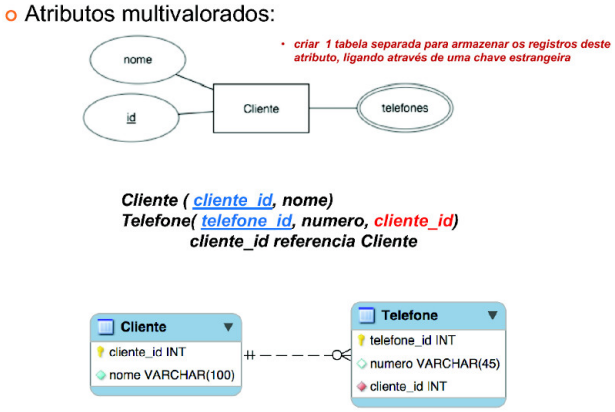


Figura 18: Exemplo com atributo multi-valorado (ou seja que pode ter muitos valores para uma mesma instância.

Comentário Nesse caso, temos a condição de que um cliente pode ter 0 ou vários telefones. Repare que notação "pé-de-galinha"mostra do lado do cliente "apenas um"e do lado do telefone, "0 ou vários".

Cllique Aqui para rever as relações da notação James Martin (pé-de-galinha).

4.4 Etapa 5- Mapeando Relacionamentos

Existem três maneiras de se mapear os relacionamentos

1. Criar Tabela Própria
    - Indicado para cardinalidades N:N em ambos os lados.
    - Criar uma tabela para o relacionamento.
    - Inserir atributo(s) do relacionamento.
    - A chave primária é formada pela concatenação das chaves de todas as entidades envolvidas no relacionamento.
2. Colunas Adicionais
    - Indicado para cardinalidades 1:N e 1:1.
    - Incluir chave estrangeira e atributos na tabela correspondente à entidade que desempenha o papel com cardinalidade máxima n.
3. Fusão de Tabelas
    - Indicado para cardinalidades 1:1 em ambos os lados.
    - Substituir as duas entidades originais por uma única entidade.
    - Escolher uma das identificações como chave primária

5 Exemplos de Mapeamento de Relacionamentos

5.1 Mapeamento de Relacionamento 1:1

Nas figuras 19 a 21 dadas a seguir temos os exemplos de mapeamento de Entidade-Relacionamento para o caso 1:1.

Para esse tipo de relacionamento temos duas opções para a criação do modelo lógico:

1. Colunas Adicionais: manter as duas tabelas das entidades 1 e 2 do relacionamento, relacionando-as a partir da relação de chaves (PK-FK).
2. Criar uma única tabela, com os atributos das duas entidades, escolhendo uma chave primária

Mostramos também um exmeplo com um relacionamento com atributo. Nesse caso podemos ter uma terceira tabela, que contempla o atributo do relacionamento.

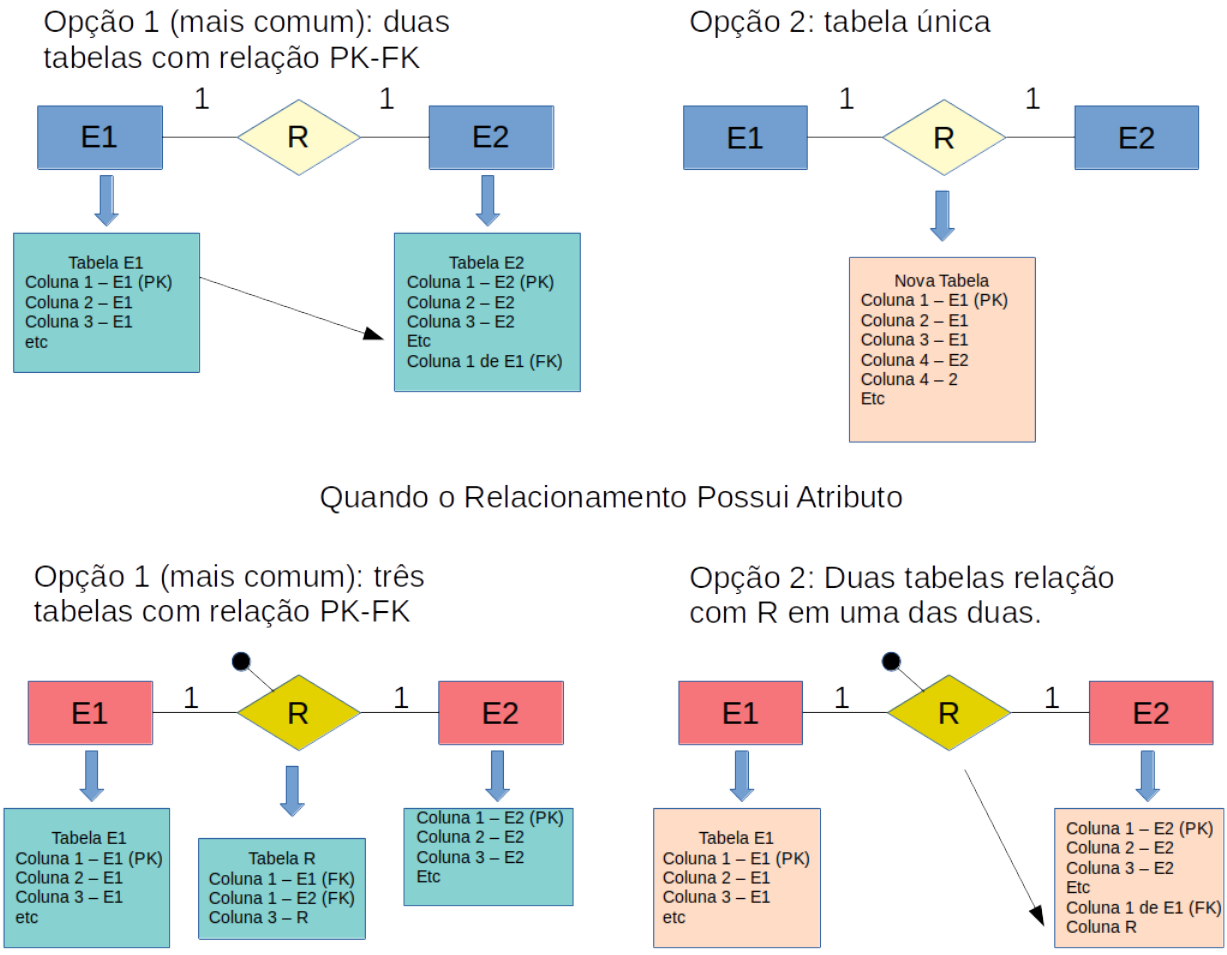


Figura 19: Diagrama ilustrando as condições do mapeamento 1:1

No exemplo da figura 20 temos o caso em que cada entidade se torna uma tabela no modelo lógico.

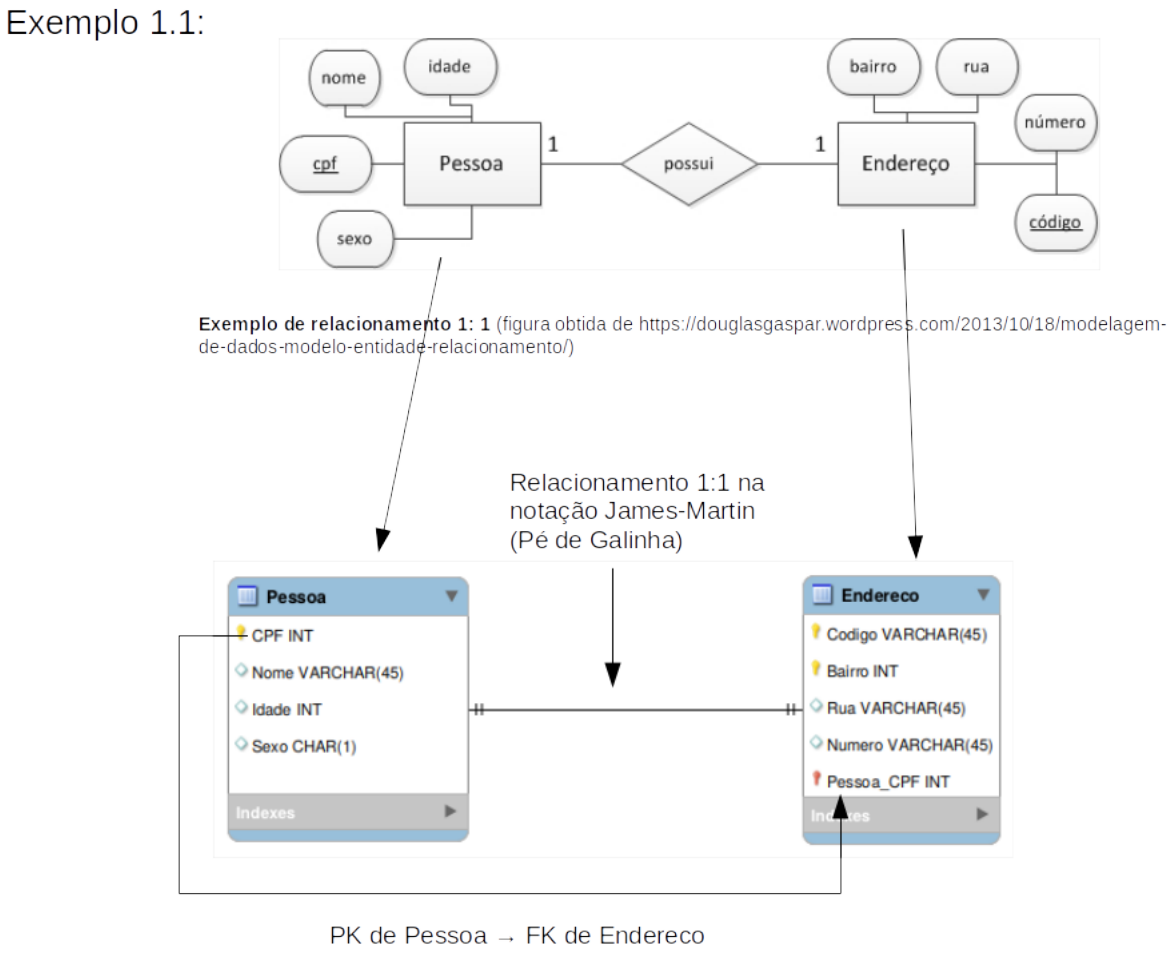
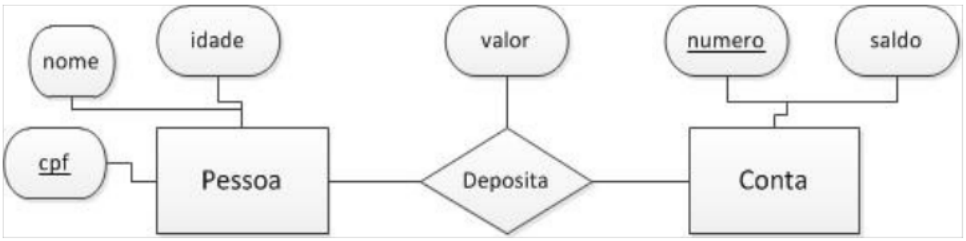


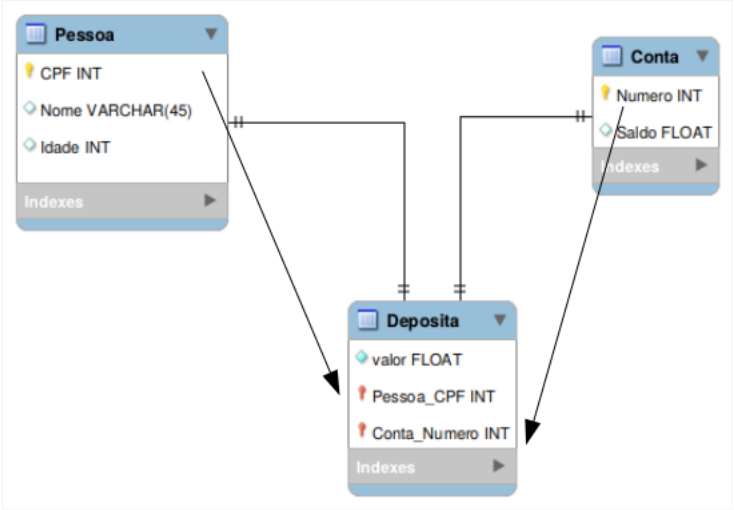
Figura 20: Exemplo de um mapeamento 1:1 do modelo conceitual para o modelo lógico

No exemplo da figura 21 temos o caso em que cada entidade se torna uma tabela no modelo lógico.

Exemplo 1.2:



Exemplo de relacionamento atributo (1: 1) (figura obtida de https://douglasgaspar.wordpress.com/2013/10/18/modelagem-de-dados-modelo-entidade-relacionamento/). Nesse exemplo temos que cada pessoa só pode ter uma conta e que cada conta está associada a uma pessoa. O relacionamento DEPOSITA possui o atributo valor.



Uma opção para esse relacionamento é criar a tabela DEPOSITA e nela inserir duas chaves estrangeiras

Figura 21: Exemplo de um mapeamento 1:1 em que o relacionamento se torna uma tabela

Vide o Exercício 1 sobre esses exemplos do mapeamento 1:1.

5.2 Mapeamento de Relacionamento 1:N

Nas figuras 22 a 24 temos os exemplos de mapeamento de Entidade-Relacionamento para o caso 1:N.

Para o caso 1:N, quando não há atributo na relação, usa-se a opção de duas tabelas, com a relação PK-FK entre elas. Geralmente o "lado N"conterá a chave estrangeria (FK) da chave primária ("lado 1").

No caso em que o relacionamento possui atributos, temos duas opções:

1. Colunas Adicionais: manter as duas tabelas das entidades 1 e 2 do relacionamento, relacionando-as a partir da relação de chaves (PK-FK).
2. Criar uma terceira tabela, que relaciona as entidades 1 e 2 do relacionamento.

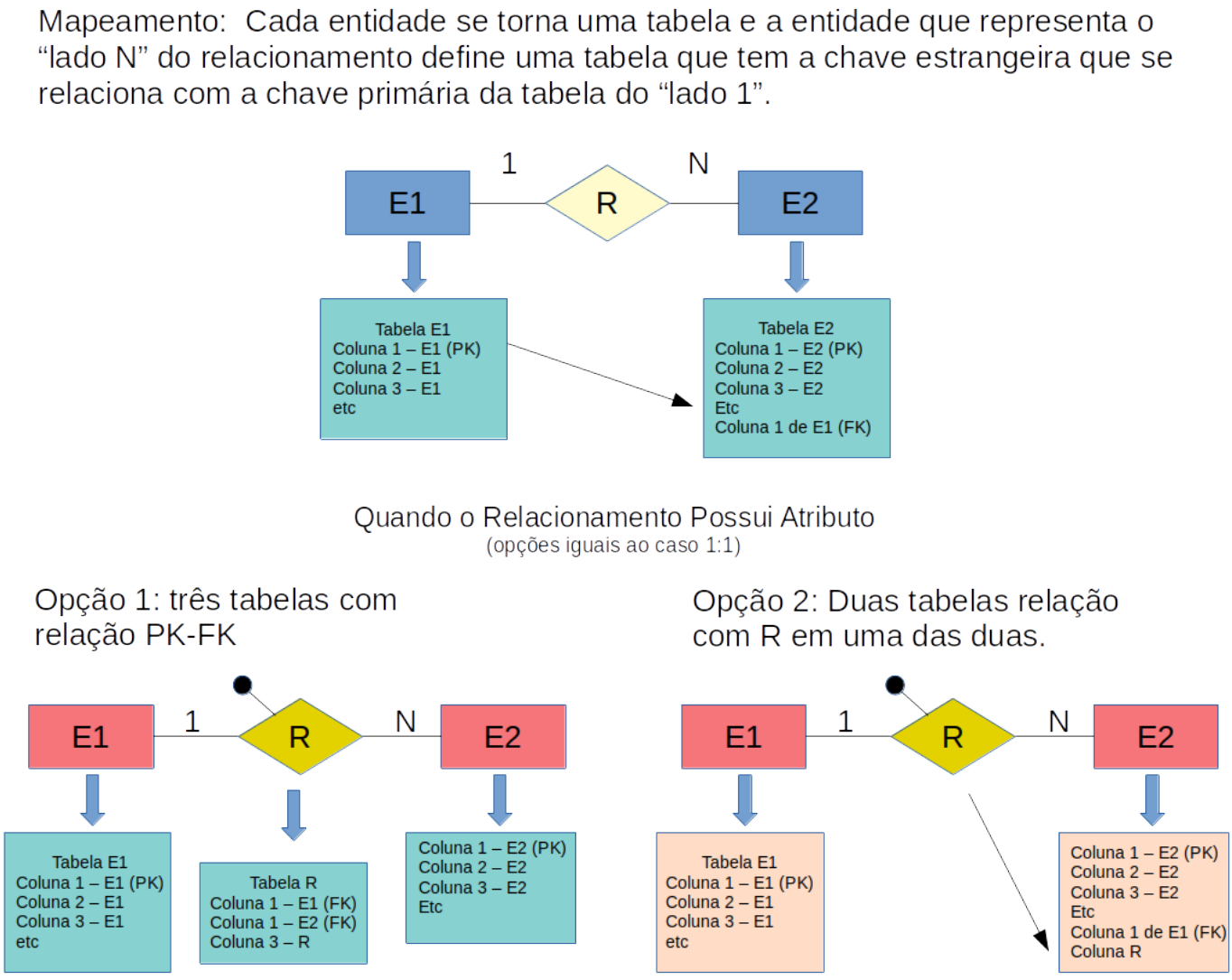


Figura 22: Exemplo de um mapeamento 1:N

Exemplo 2.1: Relacionamento 1: N sem atributo na relação

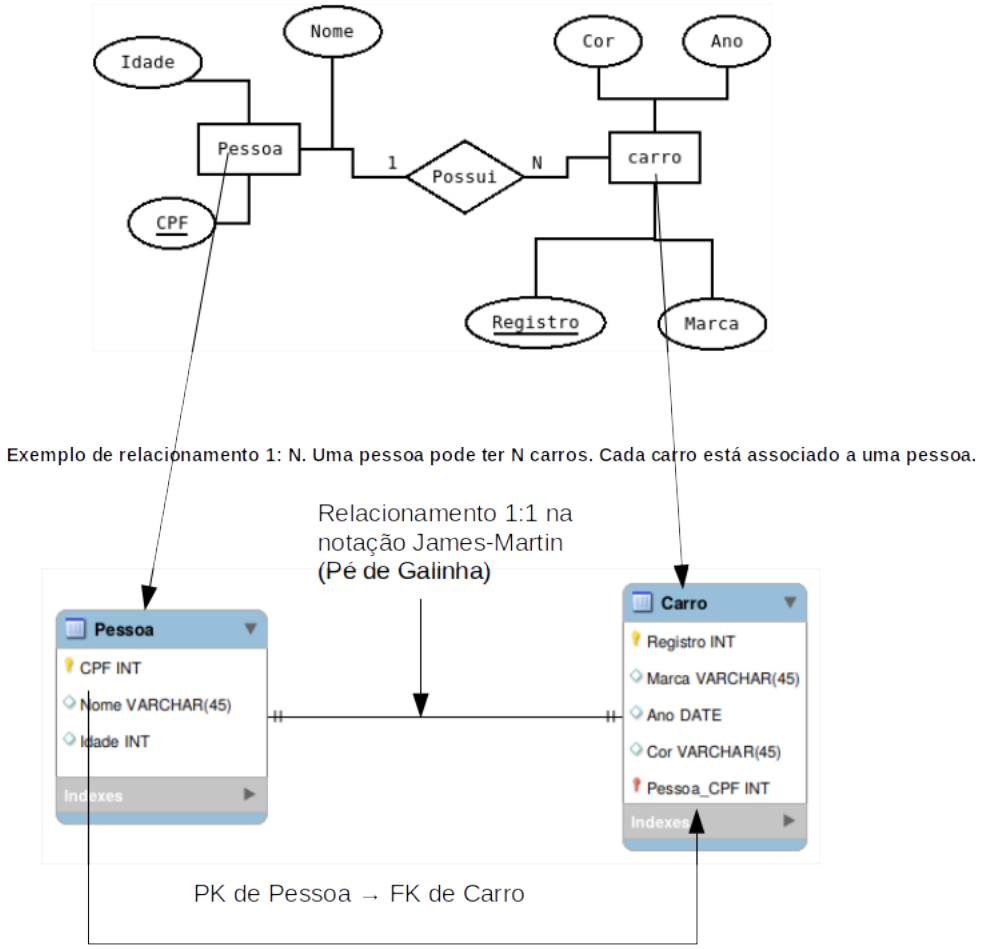
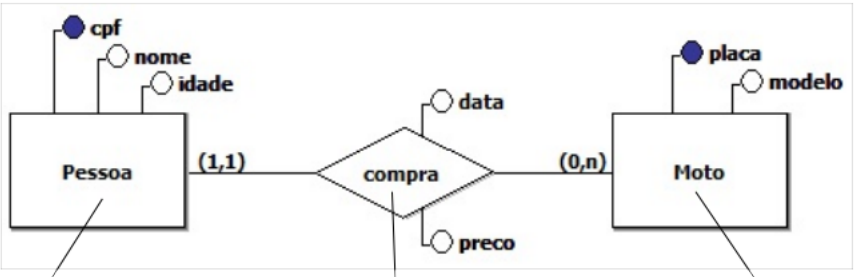
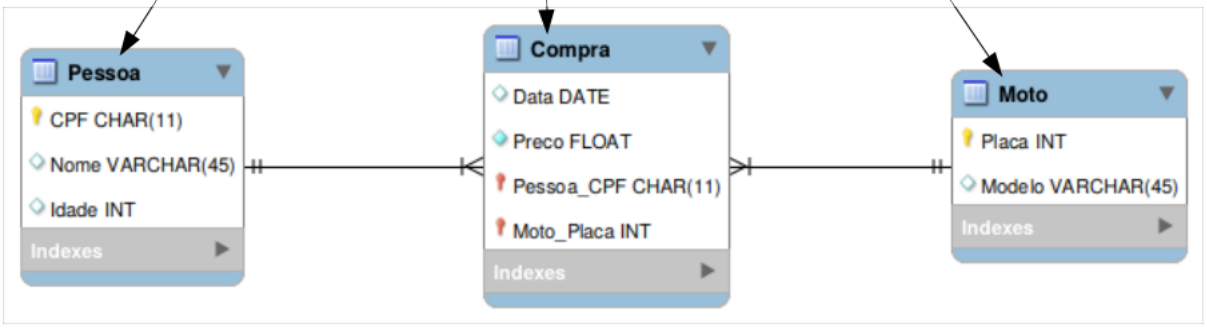


Figura 23: Exemplo de um mapeamento 1:N

Exemplo 2.2: Relacionamento 1:N com atributo na relação.



Exemplo de relacionamento 1: N com atributo. Usando a notação Chen padrão. Mostando cardinalidades máximas e mínimas.
Figura obtida de http://spaceprogrammer.com/bd/regras-de-mapeamento-esquema-er-para-logico-relacional/.



- Uma pessoa pode comprar várias motos.
- Uma moto é comprada por uma pessoa.
- Uma compra tem como campos: data e preço e informação da pessoa que está comprando e da moto que está sendo comprada.
- A moto pode ser vendida, em momentos diferentes, para diferenets pessoas. Ou seja a relação na tabela compra: Pessoa_CPF-Moto_Placa, pode existir para diferentes pessoas e diferentes motos, mas o par tem que ser único.

Figura 24: Exemplo de um mapeamento 1:N

Vide o Exercício 3 sobre os esses exemplos do mapeamento 1:N.

5.3 Mapeamento de relacionamento N:N

Nas figuras 25 a 27 temos exemplos do mapeamento do modelo conceitual para o lógico no caso de relacionamentos N:N.

Mapeamento N:N (com ou sem atributos)

Mapeamento: Cada entidade (E1 e E2 no exemplo abaixo) se torna uma tabela e o relacionamento (R no exemplo abaixo) também se torna uma tabela". Caso o relacionamento tenha atributos, eles serão acrescentados a tabela R

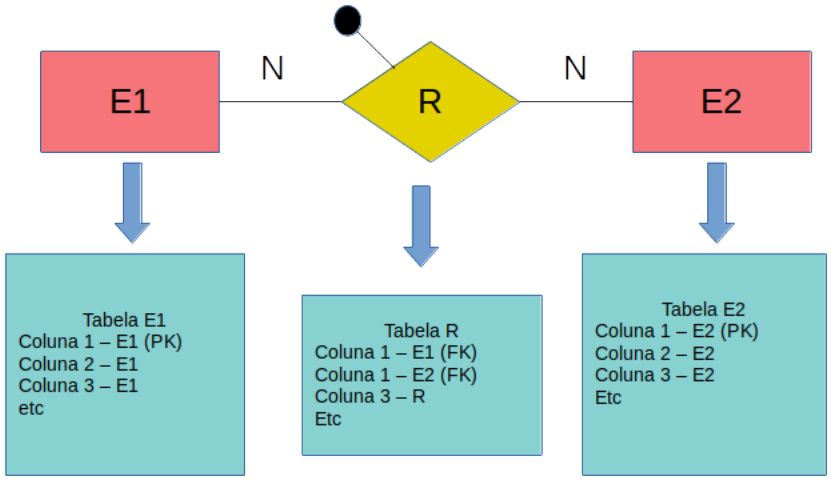


Figura 25: Exemplo de um mapeamento N:N

Exemplo 2.1: Relacionamento N: N sem atributo na relação

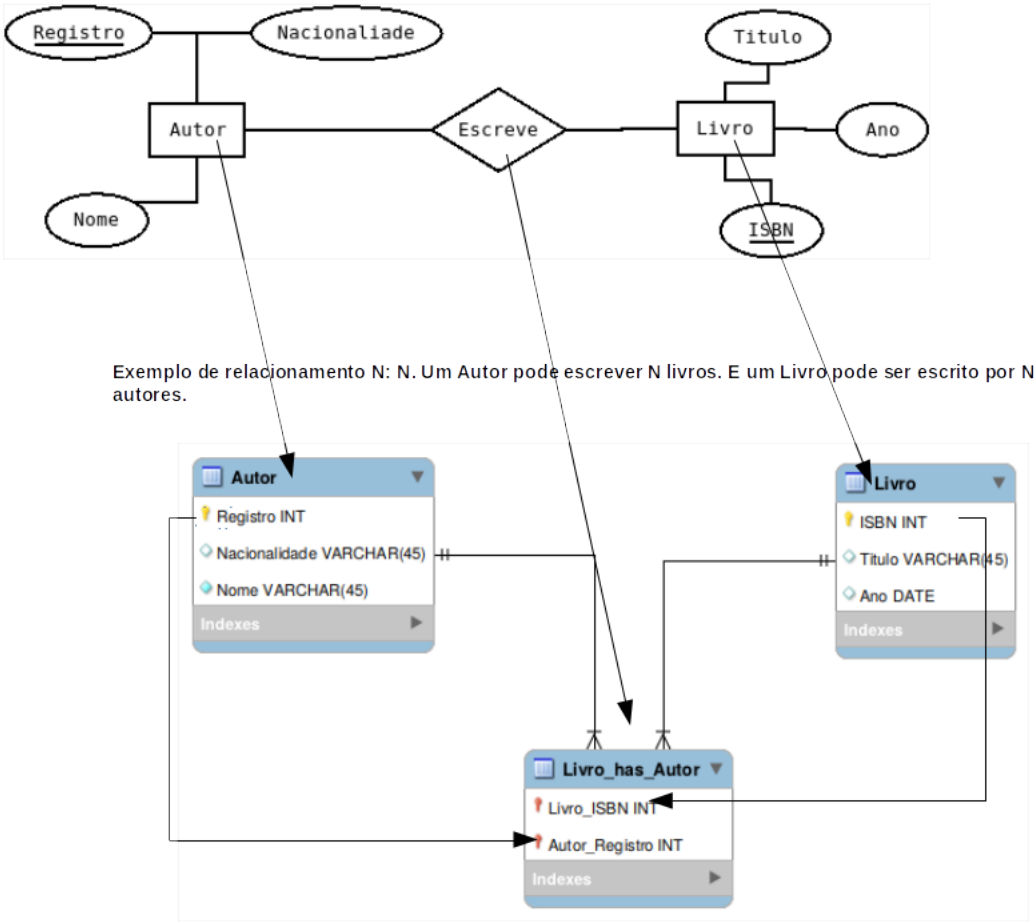
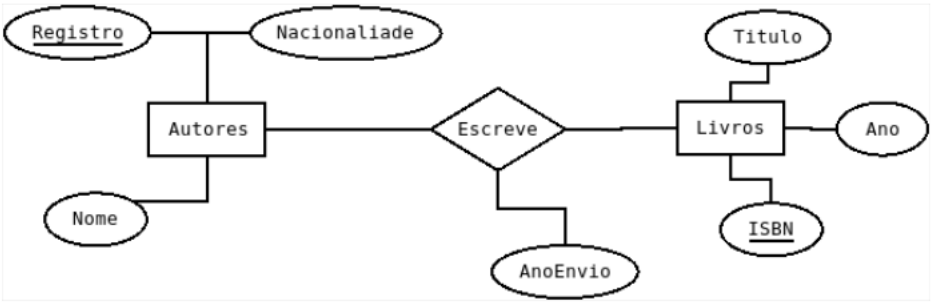


Figura 26: Exemplo de um mapeamento N:N

Exemplo 2.2: Relacionamento N:N com atributo na relação.



Exemplo de relacionamento N: N com atributo.

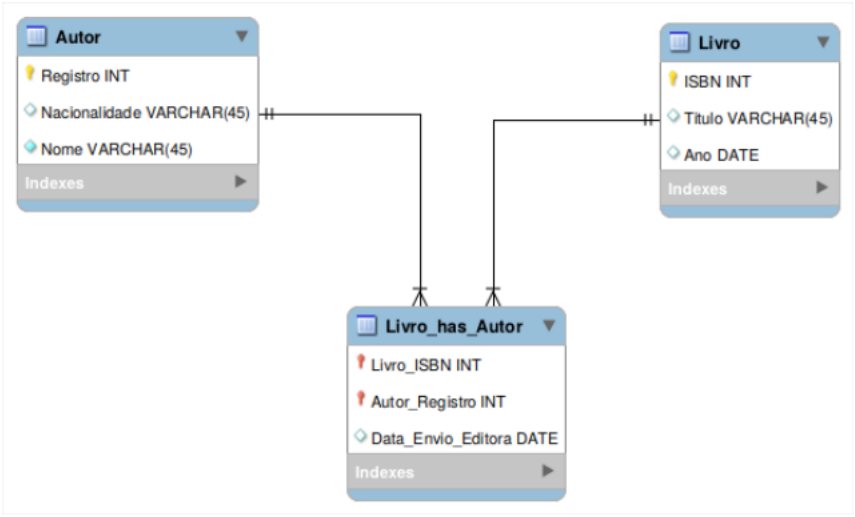


Figura 27: Exemplo de um mapeamento N:N

Vide o Exercício 4 sobre os esses exemplos do mapeamento 1:N.

6 Exercícios

Nos exercícios que se seguem, vamos converter o modelo conceitual para o modelo lógico.

O que (e como) fazer:

1. Fazer o modelo conceitual (DER) no br-modelo
2. No próprio br-modelo converter para o modelo relacional (modelo lógico)
3. Fazer o modelo lógico no MysqlWorkBench
4. Opcional (por enquanto): exportar para o SGBD (mysql)

Exercício 1 Para os dois casos do mapeamento de relacionamento 1:1 dados na subseção 5.1, a saber o exemplo do DER PESSOA-ENDEREÇO e PESSOA-CONTA, apresente o modelo lógico relacional para os casos não dados nos exemplos, ou seja, o caso "tabela única"no exemplo PESSOA-ENDEREÇO e o caso "tabela com relacionamento"no caso PESSOA-CONTA.

Você pode usar ferramentas online de modelagem ou (preferencialmente) o MysqlWorkbench.

Exercício 2 Crie exemplos diferentes dos dados no exercício 1 acima para relacionamentos 1:1 e 1:N. Para ambos faça:

- Modelo usando DER. Para relacionamento binário com e sem atributo.
- Modelo lógico a partir do DER criado.

Apresente os diagramas ER e as tabelas do modelo relacional.

Você pode usar ferramentas online de modelagem ou (preferencialmente) o MysqlWorkbench.

Exercício 3 Estude os exemplos do mapeamento para o relacionamento 1:N dados na seção 5.2.

- Responda: Para o exemplo 2.1 é possível construir um modelo lógico com uma única tabela (fusão das duas tabelas)? Sim/Não/Explique. Se sim, construa tabela e dê exemplos (com instâncias de entidades). Justifique sua resposta.
- Para o exemplo 2.2, pede-se:
  - É possível fazer um modelo relacional com duas tabelas (e não 3, como no exemplo)? Sim/Não/Explique
  - Caso seja possível, construa o modelo relacional com duas tabelas. Exemplifique com valores (instâncias das entidades).

Apresente os diagramas ER e as tabelas do modelo relacional.

Você pode usar ferramentas online de modelagem ou (preferencialmente) o MysqlWorkbench.

Exercício 4 - Desafio (Requer conhecimento do MysqWorkBench e das restrições das chaves primárias e estrangeiras).

Apesar da fase do modelo lógico não resultar na construção do Banco de Dados no SGBD, vamos nesse exercício nos antecipar ao estudo do modelo físico, fazendo uma implementação de um modelo lógico (que a rigor tem que ser convertido no modelo físico) no Mysql.

Para isso, vamos implementar o modelo lógico dado no Exemplo 2.2 cujo diagrama no "padrão"do MysqlWorkBench é repetido abaixo na figura 28.

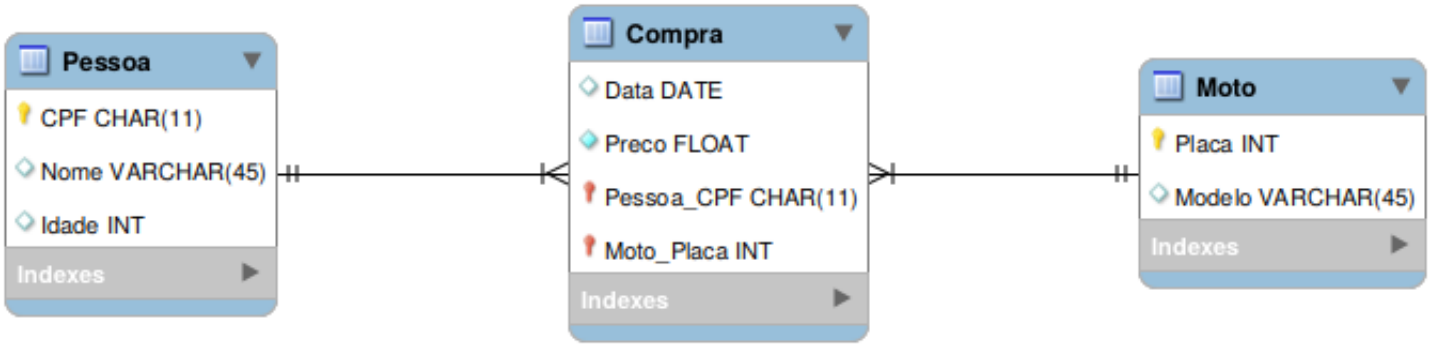


Figura 28: Diagrama para Exercício 4

Para esse diagrama, pede-se:

1. Crie as tabelas do Banco no Mysql
2. Insira dados para as tabelas PESSOA e MOTO
3. Insira dados na tabela compra, representando instâncias da entidade PESSOAS comprando motos, que são instâncias da entidade MOTO.
4. Analise a inserção na tabela COMPRA em relação ao par de chaves nela presentes.
5. Entenda a(s) restrição(ções) criadas pela relação 1:N
6. Analise e responda: é possível, do ponto de vista de inserção coreta dos dados e manutenção das relações, usar para esse caso um relacionamento N:N? SIM/NÃO/Explique. Se sim, dê exemplos.

7 Referências

- Artigo sobre mapeamento DER-Modelo Lógico: Mapeamento de Modelos ER para DDL da SQL
- Mapeamento DER-Lógico Mapeamento de Modelos ER Modelo Lógico
- Mapeamento DER-Modelo Lógico: ER-to-Relational Mapping
- Mapeamento DER-Modelo Lógico: Mapeamento de Modelo ER-Lógico

Compilado 19 de outubro de 2026 - 18:05:58- fantini@ufmg.br